

ASSIGNMENT-9.1

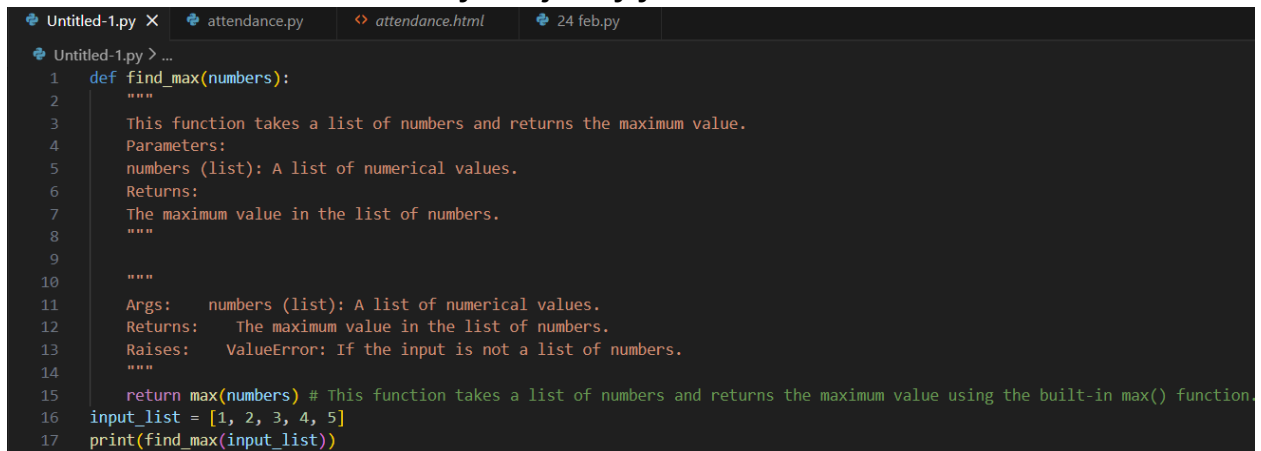
Name: N.Trinandini
Hall Ticket No:2303A52429
BATCH :32

Consider the following Python function:

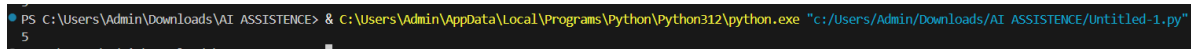
```
def find_max(numbers):  
    return max(numbers)
```

Task:

- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer



```
Untitled-1.py X attendance.py attendance.html 24 feb.py  
1 def find_max(numbers):  
2     """  
3     This function takes a list of numbers and returns the maximum value.  
4     Parameters:  
5     numbers (list): A list of numerical values.  
6     Returns:  
7     The maximum value in the list of numbers.  
8     """  
9  
10    """  
11    Args:    numbers (list): A list of numerical values.  
12    Returns: The maximum value in the list of numbers.  
13    Raises:  ValueError: If the input is not a list of numbers.  
14    """  
15    return max(numbers) # This function takes a list of numbers and returns the maximum value using the built-in max() function.  
16    input_list = [1, 2, 3, 4, 5]  
17    print(find_max(input_list))
```



```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/Untitled-1.py"  
5  
5  
PS C:\Users\Admin\Downloads\AI ASSISTENCE>
```

Inline Comments

Advantages:

- Simple and quick.
- Useful for explaining specific lines.

Disadvantages:

- No structure.
- Cannot clearly document parameters or exceptions.
- Not suitable for reusable libraries.

Best Use Case:

Small scripts or explaining complex logic inside a function.

Basic Docstring

Advantages:

- Clear overview of function purpose.
- Easy to write and understand.

Disadvantages:

- No strict structure.
- Exceptions are often unclear.

Best Use Case:

Small projects or beginner-level code.

Google-Style Docstring

Advantages:

- Structured and consistent.

- Clearly documents arguments, return types, and exceptions.
- Suitable for auto-generated documentation tools.

Disadvantages:

- Slightly more verbose.

Best Use Case:

Professional projects, libraries, APIs, and large systems.

Use Google-style documentation.

Reason: A mathematical utilities library will be reused by many developers. Functions must clearly define input types, return types, and possible errors. Structured documentation prevents misuse and improves maintainability.

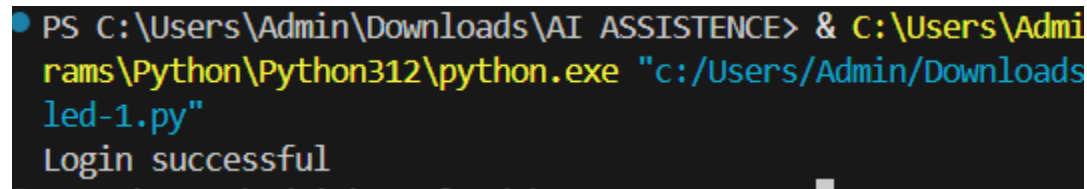
Problem 2: Consider the following Python function:

```
def login(user, password, credentials):  
  
    return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

```
def login(user,password,credentials):
    """
    This function takes a username, password, and a dictionary of credentials and checks if the username and password match any entry in the credentials dictionary.
    Parameters:
    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.
    Returns:
    A string indicating whether the login was successful or not.
    """
    """
    Args:
    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.
    Returns:
    A string indicating whether the login was successful or not.
    Raises:
    KeyError: If the username is not found in the credentials dictionary.
    """
    if not isinstance(credentials, dict):
        raise ValueError("Credentials must be a dictionary")
    if user in credentials and credentials[user] == password:# This function checks if the provided username exists in the credentials
        #dictionary and if the corresponding password matches the provided password.
        return "Login successful"
    else:
        return "Login failed"
input_credentials = {"user1": "password1", "user2": "password2"}
print(login("user1", "password1", input_credentials))
```



```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\Downloads\Python\Python312\python.exe "c:/Users/Admin/Downloads/led-1.py"
Login successful
```

Inline Comments

Advantages:

- Very simple.
- Useful for explaining a single line of logic.

Disadvantages:

- No structure.
- Does not clearly define inputs or return type.
- Not helpful for understanding function contract.

Use Case:

Small scripts or quick internal code.

Basic Docstring

Advantages:

- Gives a clear description of purpose.
- Easy to write and read.

Disadvantages:

- No strict structure.
- Exceptions and return types may not be clearly defined.

Use Case:

Small or medium projects.

Google Style**Advantages:**

- Clearly separates arguments, returns, and errors.
- Easy to scan and consistent.
- Good for team projects.

Disadvantages:

- Slightly longer to write.

Use Case:

Professional systems, APIs, shared libraries.

Use Google-style documentation.

Reason: New developers need clarity about inputs, outputs, and possible errors. Structured documentation reduces confusion and speeds up onboarding.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation.

Instructions:

- 1. Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:**
 - o `add(a, b)` – returns the sum of two numbers**
 - o `subtract(a, b)` – returns the difference of two numbers**
 - o `multiply(a, b)` – returns the product of two numbers**
 - o `divide(a, b)` – returns the quotient of two numbers**
- 2. Display the module documentation in the terminal using Python's documentation tools.**
- 3. Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.**

```

def add(a,b):
    """
    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    return a+b
def subtract(a,b):
    """
    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    return a-b
def division(a,b):
    """
    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    if b==0:
        return "division by zero is not allowed"
    else:
        return a/b
print(add.__doc__)
print(subtract.__doc__)
print(division.__doc__)

```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/calculatorexample.py"

```

```

    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE>

```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc calculator

this function takes two numbers as input and returns their sum
Parameters:
a (int or float): the first number
b (int or float): the second number

this function takes two numbers as input and returns their difference
Parameters:
a (int or float): the first number
b (int or float): the second number

this function takes two numbers as input and returns their quotient
Parameters:
a (int or float): the first number
b (int or float): the second number

Help on module calculator:

NAME
calculator

DESCRIPTION
this module include four function and
their implementation
and also includes a docstring to explain the usage of each function

FUNCTIONS
add(a, b)
    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

division(a, b)
    this function takes two numbers as input and returns their quotient
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc calculator
```

```
  a (int or float): the first number  
  b (int or float): the second number
```

```
subtract(a, b)
```

```
  this function takes two numbers as input and returns their difference
```

```
Parameters:
```

```
  a (int or float): the first number  
  b (int or float): the second number
```

```
FILE
```

```
c:\users\admin\downloads\ai assistance\calculator.py
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE>
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE>
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w calculator
```

```
  this function takes two numbers as input and returns their sum
```

```
Parameters:
```

```
  a (int or float): the first number  
  b (int or float): the second number
```

```
  this function takes two numbers as input and returns their difference
```

```
Parameters:
```

```
  a (int or float): the first number  
  b (int or float): the second number
```

```
  this function takes two numbers as input and returns their quotient
```

```
Parameters:
```

```
  a (int or float): the first number  
  b (int or float): the second number
```

```
wrote calculator.html
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 2429
```

```
Server ready at http://localhost:2429/
```

```
Server commands: [b]rowser, [q]uit
```

```
server> b
```

```
server> █
```

Index of Modules

Built-in Modules

_abc	_imp	_statistics	builtins
_ast	_io	_string	cmath
_bisect	_json	_struct	errno
_blake2	_locale	_symtable	faulthandler
_codecs	_lspref	_thread	gc
_codecs_cn	_md5	_tokenize	itertools
_codecs_hk	_multibytecodec	_tracemalloc	marshal
_codecs_iso2022	_opcode	_typing	math
_codecs_jp	_operator	_warnings	mmap
_codecs_kr	_pickle	_weakref	mmap
_codecs_tw	_random	_winapi	mmap
_collections	_sha1	_xxinterchannels	mmap
_contextvars	_sha2	_xxsubinterpreters	mmap
_csv	_sha3	array	mmap
_datetime	_signal	atexit	mmap
_functools	_sre	audioop	mmap
_heapq	_stat	binascii	mmap

C:\Users\Admin\Downloads\AI ASSISTENCE

20th	aiac	even numbers	lab_8
24 feb	aiac_feb.17	feb.18	lab_ass_8
Untitled-1	attendance	jan.13th	labexam_2
Untitled-2	calculator	lab-7	labexamaia

C:\Users\Admin\AppData\Local\Programs\Python\Python312\python312.zip

afxres

[index](#)
[c:\users\admin\appdata\local\programs\python\python312\site-packages\win32\lib\afxres.py](#)

Generated by h2py from stdin

Functions

[AFX_CONTROLBAR_MASK\(nIDC\)](#)

Data

[AFX_IDB_CHECKLISTBOX_95](#) = 30996
[AFX_IDB_CHECKLISTBOX_NT](#) = 30995
[AFX_IDB_MINIFRAME_MENU](#) = 30994
[AFX_IDB_TRUETYPE](#) = 32384
[AFX_IDC_BROWSE](#) = 1203
[AFX_IDC_CHANGE](#) = 101
[AFX_IDC_CLEAR](#) = 1204
[AFX_IDC_COLORPROP](#) = 1116
[AFX_IDC_COLOR_BLACK](#) = 1100
[AFX_IDC_COLOR_BLUE](#) = 1104
[AFX_IDC_COLOR_CYAN](#) = 1107
[AFX_IDC_COLOR_DARKBLUE](#) = 1112
[AFX_IDC_COLOR_DARKCYAN](#) = 1115
[AFX_IDC_COLOR_DARKGREEN](#) = 1111
[AFX_IDC_COLOR_DARKMAGENTA](#) = 1114
[AFX_IDC_COLOR_DARKRED](#) = 1110
[AFX_IDC_COLOR_GRAY](#) = 1108
[AFX_IDC_COLOR_GREEN](#) = 1103
[AFX_IDC_COLOR_LIGHTBROWN](#) = 1113
[AFX_IDC_COLOR_LIGHTGRAY](#) = 1109
[AFX_IDC_COLOR_MAGENTA](#) = 1106
[AFX_IDC_COLOR_RED](#) = 1102
[AFX_IDC_COLOR_WHITE](#) = 1101

Problem 4: Conversion Utilities Module

Task:

- Write a module named `conversion.py` with functions:
 - `decimal_to_binary(n)`
 - `binary_to_decimal(b)`
 - `decimal_to_hexadecimal(n)`
- Use Copilot for auto-generating docstrings.
- Generate documentation in the terminal.
- Export the documentation in HTML format and open it in a browser.

```

def decimal_to_binary(n):
    """
    This function takes a decimal number as input and returns its binary representation as a string
    """
    if n == 0:
        return "0"
    binary = ""
    while n > 0:
        binary = str(n % 2) + binary
        n = n // 2
    return binary
print(decimal_to_binary(10)) # Output: 1010
print(decimal_to_binary(0)) # Output: 0
def binary_to_decimal(b):
    """
    This function takes a binary number as input and returns its decimal representation as an integer
    """
    decimal = 0
    for i in range(len(b)):
        decimal += int(b[-(i+1)]) * (2 ** i)
    return decimal
print(binary_to_decimal("1010")) # Output: 10
print(binary_to_decimal("0")) # Output: 0
def decimal_to_hexadecimal(n):
    """
    This function takes a decimal number as input and returns its hexadecimal representation as a string
    """
    if n == 0:
        return "0"
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    while n > 0:
        hexadecimal = hex_digits[n % 16] + hexadecimal
        n = n // 16
    return hexadecimal
print(decimal_to_hexadecimal(255)) # Output: FF
print(decimal_to_hexadecimal(0)) # Output: 0

```

Ln 40, Col 38 Spaces: 4 UTF-8 CRLF {} Python

```

def decimal_to_hexadecimal(n):
    """
    This function takes a decimal number as input and returns its hexadecimal representation as a string
    """
    if n == 0:
        return "0"
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    while n > 0:
        hexadecimal = hex_digits[n % 16] + hexadecimal
        n = n // 16
    return hexadecimal
print(decimal_to_hexadecimal(255)) # Output: FF
print(decimal_to_hexadecimal(0)) # Output: 0
print(binary_to_decimal.__doc__)
print(decimal_to_binary.__doc__)
print(decimal_to_hexadecimal.__doc__)

```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/conversion.py"
1010
0
10
0
FF
0

This function takes a binary number as input and returns its decimal representation as an integer

This function takes a decimal number as input and returns its binary representation as a string

This function takes a decimal number as input and returns its hexadecimal representation as a string
PS C:\Users\Admin\Downloads\AI ASSISTENCE>

```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc conversion
1010
0
10
0
FF
0

This function takes a binary number as input and returns its decimal representation as an integer

This function takes a decimal number as input and returns its binary representation as a string

This function takes a decimal number as input and returns its hexadecimal representation as a string
Help on module conversion:

NAME
    conversion

FUNCTIONS
    binary_to_decimal(b)
        This function takes a binary number as input and returns its decimal representation as an integer

    decimal_to_binary(n)
        This function takes a decimal number as input and returns its binary representation as a string

    decimal_to_hexadecimal(n)
        This function takes a decimal number as input and returns its hexadecimal representation as a string

FILE
    c:\users\admin\downloads\ai assistance\conversion.py

```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w conversion
1010
0
10
0
FF
0

```

This function takes a binary number as input and returns its decimal representation as an integer

This function takes a decimal number as input and returns its binary representation as a string

This function takes a decimal number as input and returns its hexadecimal representation as a string

wrote conversion.html

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 2429
Server ready at http://localhost:2429/
Server commands: [b]rowser, [q]uit
server> b
server>

```

Python 3.12.5 [tags/v3.12.5:f3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

[Module Index](#) : [Topics](#) : [Keywords](#)

Index of Modules

Built-in Modules

_abc	_imp	_statistics	builtins
_ast	_io	_string	cmath
_bisect	_json	_struc	errno
_blake2	_locale	_symtable	faulthandler
_codecs	_logprof	_thread	gc
_codecs_cn	_md5	_tokenize	itertools
_codecs_hk	_multibytecodec	_tracemalloc	marshal
_codecs_iso2022	_opcode	_typing	math
_codecs_jp	_operator	_warnings	mmap
_codecs_kr	_pickle	_weakref	msvcrt
_codecs_tw	_random	_winapi	nt
collections	_sha1	_xxinterpchannels	os
contextvars	_sha2	_xxsubinterpreters	sys
csv	_sha3	array	time
datetime	_signal	atexit	winreg
functools	_sre	audioop	xxsubtype
heapq	_stat	binascii	zlib

C:\Users\Admin\Downloads\AI ASSISTENCE

20th	aiac	even numbers	lab_8
24 feb	aiac_feb.17	feb_18	lab_ass_8
Untitled-1	attendance	jan.13th	labexam_2
Untitled-2	conversion	lab-7	labexamaiac

C:\Users\Admin\AppData\Local\Programs\Python\Python312\python312.zip

dataclasses

Modules

[_thread](#)
[abc](#)
[copy](#)

[functools](#)
[inspect](#)
[itertools](#)

[keyword](#)
[re](#)
[sys](#)

Classes

[builtins.AttributeError\(builtins.Exception\)](#)
[FrozenInstanceError](#)
[builtins.object](#)
[Field](#)
[InitVar](#)

class **Field**([builtins.object](#))

[Field](#)(default, default_factory, init, repr, hash, compare, metadata, kw_only)

```
# Instances of Field are only ever created from within this module,  
# and only from the field\(\) function, although Field instances are  
# exposed externally as (conceptually) read-only objects.  
#  
# name and type are filled in after the fact, not in __init__.  
# They're not known at the time this class is instantiated, but it's  
# convenient if they're available later.  
#  
# When cls._FIELDS is filled in with a list of Field objects, the name  
# and type fields will have been populated.
```

Methods defined here:

```
__init__(self, default, default_factory, init, repr, hash, compare, metadata, kw_only)  
    Initialize self. See help(type(self)) for accurate signature.
```